
B28: SQL SuperHero Program : Session 4

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>

17 February 2025 at 05:51

Bcc: satishhattekar1997@gmail.com

What is a foreign key (referential integrity) ?

FK is for parent-child relationships.

A foreign key is a way to enforce referential integrity within your Oracle database. A foreign key means that values in one table must also appear in another table.

The referenced table is called the parent table while the table with the foreign key is called the child table. The foreign key in the child table will generally reference a primary key in the parent table.

--> if there is a child exists, there must be a parent exists

Why we need FK:

To establish referential integrity

Foreign key constraint to enforce a business rule.

--> suppose we need to establish a parent child relationship, like if data exists in child table, there must be a parent table where we have details. if we have to enforce this rule, we need FK.

suppose we have 2 tables, 1.item and 2.parts .

Item table contains : fan, tubelight etc records

parts table contains : parts of fan, parts of tubelight (it only contains the parts of item which are available in database)

--> we do not want any such data in parts tables, for which item not exists. Suppose we are not selling AC, then I do not want AC parts to be in parts table.

when we try to insert in parts table as AC as item, it gives referential integrity error as the parent not exists.

```
CREATE TABLE supplier
( supplier_id number(10) primary key,
  supplier_name varchar2(50) not null,
  contact_name varchar2(50)
);
```

```
CREATE TABLE products
( product_id number(10) not null,
  supplier_id number(10) not null
);
```

```
ALTER TABLE products
```

```
ADD CONSTRAINT fk_vishal -- any name u can write fk_vishal
```

```
FOREIGN KEY (supplier_id)
```

```
REFERENCES supplier(supplier_id);
```

Alter table <tablename> ADD CONSTRAINT <constraint name> FOREIGN KEY <column name> REFERENCES <parent table>(<parent column name>)

--> parent table: supplier

--> child table --> products

```
insert into supplier values (1, 'azx','yryryr');
insert into supplier values (2, 'rrr','hyuj');
insert into supplier values (3, 'ggg','seedf');
insert into supplier values (4, 'uuu','ghyujk');
```

```
select * from supplier;
```

```
insert into products values (1,1);
insert into products values (1,4);
```

insert into products values (4,5); --> 5 is not exists in supplier table as supplier id , so violates referential integrity and gives error (parent key not found)

5 not present in supplier table

```
insert into product values (4,5)
```

ORA-02291: integrity constraint (SQL_FEHSCAHEETNUTKNQIDIOAGDN.VISHAL_FK) violated - parent key not found ORA

More Details: <https://docs.oracle.com/error-help/db/ora-02291>

it gives error

if we insert a row in master table(parent table)

insert into supplier values (5, 'uuu','ghyujk');

then try to insert in child table:

insert into products values (1,5); --> **Now it should insert because we have data in the parent table.**

```
insert into supplier values (5, 'erfrt')
```

1 row(s) inserted.

→ insert into Parent first

```
insert into products values (2, 5)
```

1 row(s) inserted.

→ Now we can insert into child

=====

A foreign key can be defined in either a CREATE TABLE statement or an ALTER TABLE statement.

```
CREATE TABLE products
( product_id number(10) not null,
  supplier_id number(10) not null,
  CONSTRAINT fk_vishal
    FOREIGN KEY (supplier_id)
    REFERENCES supplier(supplier_id)
);
```

```
ALTER TABLE products
ADD CONSTRAINT fk_vishal
  FOREIGN KEY (supplier_id)
  REFERENCES supplier(supplier_id);
```

=====

Can I Insert Null Into FK?

YES.

```
insert into products values (11,null)
```



```
1 row(s) inserted.
```

✓ inserted

Foreign key simply requires that the value in that field must exist first in a different table (the parent table). It can be NULL and can be duplicated.

Null by definition is not a value. Null means that we do not yet know what the value is. Most FKs are for a one to many relationship

When a foreign key's value is not known at the time of record generation then it's kept null. When the data try to insert in child and there is null value, so it will go and compare in parent but null is non comparable, so can not say that its present or not. That's why null allowed.

=====

Interview Questions in sequence :

can we drop FK?

2 options: we can drop or we can disable FK

Alter table <table name> DROP constraint <constraint name>;

Disable:

**ALTER TABLE child_table
DISABLE CONSTRAINT fk_name;**

`ALTER TABLE products DISABLE CONSTRAINT fk_vishal;`

`insert into products values (11,7);`

supplier id 7 is not exists in supplier table, but still data will insert because our FK constraint is disabled.

insert into products values (11,7)

1 row(s) inserted.

--> next question he will ask, can we enable a constraint ?

We can enable the FK also by using command:

ALTER TABLE product enable CONSTRAINT fk_vishal;

```
alter table products enable constraint fk_vishal
```

ORA-02298: cannot validate (SQL_SEUBNOXTNRQLFEOXQZYLPZEQY.FK_VISHAL) - parent keys not found

--> we can not enable, but it gives error, if any such record exists which violates the FK rule.(Means for the child , no parent exists) so in that scenario we have to enable with novalidate option

alter table products(child table) enable novalidate constraint fk_vishal;

```
alter table products enable novalidate constraint fk_vishal
```

Table altered.

Novalidate means: DO NOT CHECK PREVIOUS DATA

So when we enable the constraint with novalidate , it will not check any old data, but the constraint will work for any new records and validate the parent child relationship. after enable no validate, rule will work for new records not for old records.

Can we delete a record from a parent, if the record exists in Child?

NO

```
delete from supplier where supplier_id = 1
```

we have record present in child table.
→ so can not delete from Parent

ORA-02292: integrity constraint (SQL_DEYRIDYIXXPSNOIMRNXYBTUFE.FK_VISHAL) violated - child record found ORA-06512: .

More Details: <https://docs.oracle.com/error-help/db/ora-02292>

Reason is the same: **FK is saying that if a child exists, there must be a parent.** So how can we delete from a parent, if we do so then we left an orphan child, which is not allowed.

=====

Question: How can we delete records from multiple tables from 1 delete statement in oracle?

Answer: ON DELETE CASCADE

While creating a table, if we create a FK with ON DELETE CASCADE option, so it means, when data will be deleted from parent table, it will delete from all child tables as well.

```
CREATE TABLE supplier
( supplier_id number(10) primary key,
  supplier_name varchar2(50) not null,
  contact_name varchar2(50)
);
```

```
CREATE TABLE products
( product_id number(10) not null,
  supplier_id number(10) not null
);
```

```
insert into supplier values (1, 'azx', 'yryryr');
insert into supplier values (2, 'rrr', 'hyuj');
insert into supplier values (3, 'ggg', 'seedf');
insert into supplier values (4, 'uuu', 'ghyujk');
```

```
insert into products values (1,1);
insert into products values (1,4);
```

```
ALTER TABLE products
ADD CONSTRAINT fk_vishal -- any name u can write fk_vishal
FOREIGN KEY (supplier_id)
REFERENCES supplier(supplier_id)
on delete cascade;
```

/* In the above syntax I added ON DELETE CASCADE. It means if we delete records from parent table supplier, from child also it will be deleted.*/

```
delete from supplier where supplier_id = 1;
-- 1 row deleted, but it deletes records from child table Products as well.
select * from supplier;

select * from products;
```

```

1 delete from supplier where supplier_id = 1;
2 -- 1 row deleted, but it delete records from child table Products as well.
3 select * from supplier;
4
5 select * from products;|

```

SUPPLIER_ID	SUPPLIER_NAME	CONTACT_NAME
2	rrr	hyuj
3	ggg	seedf
4	uuu	ghyujk

Download CSV
3 rows selected.

PRODUCT_ID	SUPPLIER_ID
1	4

Download CSV

If you see in the above example, it deletes the corresponding records from the product table as well.

=====

Question: I have a FK relation between 2 tables and I want data to not be deleted from the child table. What can we do?

Answer:

```

CREATE TABLE supplier
( supplier_id numeric(10) PRIMARY KEY ,
  supplier_name varchar2(50) ,
  contact_name varchar2(50)
);

```

```

CREATE TABLE products
( product_id numeric(10) not null,
  supplier_id numeric(10)
);

```

```
alter table products add constraint fk_vishal foreign key (supplier_id) references supplier(supplier_id) on delete set null;
```

I have added in last on delete set null. So, it will just set the FK column value to null in child table and not delete the whole row.

```

insert into supplier values (1, 'azx','yryryr');
insert into supplier values (2, 'rrr','hyuj');
insert into supplier values (3, 'ggg','seedf');
insert into supplier values (4, 'uuu','ghyujk');

```

```

insert into products values (1,1);
insert into products values (1,4);

```

```

delete from supplier where supplier_id = 1
select * from products;

```

```
select * from supplier
```

SUPPLIER_ID	SUPPLIER_NAME	CONTACT_NAME
2	rrr	hyuj
3	ggg	seedf
4	uuu	ghyujk

3 rows selected.

```
select * from products
```

PRODUCT_ID	SUPPLIER_ID
1	-
1	4

2 rows selected.

supplier-id = 1 is deleted
from Parent
&
Set
Null
to
Child

set to null

1. Create table "TAB1" with one column "C" of primary key
2. Create table "TAB2" with one column "C" of primary key
3. Create foreign key on TAB1.C referring TAB2.C
4. Create foreign key on TAB2.C referring TAB1.C
5. How to populate data in both tables as given below?

TAB1	
COLUMN_NAME	DATA TYPE
C	VARCHAR2(1)

TAB2	
COLUMN_NAME	DATA TYPE
C	VARCHAR2(1)

TAB1	TAB2
A	A
B	B
C	C

See the above requirements. We have 2 tables and both are related to each other with PK and FK relation. How to insert data?

```
create table tab1 (c varchar2(5) primary key);  
create table tab2 (c varchar2(5) primary key);  
alter table tab1 add constraint fk_tab1 foreign key (c) references tab2(c);  
alter table tab2 add constraint fk_tab2 foreign key (c) references tab1(c);
```

Statement 1	<pre>create table tab1(c varchar2(5) primary key)</pre> Table created.	→ TAB 1 created
Statement 2	<pre>create table tab2(c varchar2(5) primary key)</pre> Table created.	→ TAB 2 created
Statement 3	<pre>alter table tab1 add constraint tab1_fk foreign key (c) references tab2(c)</pre> Table altered.	→ FK Record in tab1 must be in tab2
Statement 4	<pre>alter table tab2 add constraint tab2_fk foreign key (c) references tab1(c)</pre> Table altered.	→ FK Record in tab2 must be in tab1

Now, If we try to Insert record in Tab1, it will check if data exists in tab2 or not. Because of Foreign key , any data in Tab1 , must exists in Tab2. So, I am trying to insert in tab1 as value "A" , and this value not exists in Tab2, so its gives the error "Parent Key Not Found" . It is correct, because this is the FK rule. Same vice versa with Tab2 insert.

<pre>insert into tab1 values ('A')</pre>	ORA-02291: integrity constraint (SQL_DEYRIDYIXXPSNOIMRNXYBTUFE.FK_TAB1) violated - parent key not found OR
More Details: https://docs.oracle.com/error-help/db/ora-02291	
<pre>insert into tab2 values ('A')</pre>	ORA-02291: integrity constraint (SQL_DEYRIDYIXXPSNOIMRNXYBTUFE.FK_TAB2) violated - parent key not found OR
More Details: https://docs.oracle.com/error-help/db/ora-02291	

→ This error is expected.

Now, the concept of deferred constraint comes into picture.

- A deferred constraint is one that is enforced when a transaction is committed.
- A deferrable constraint is specified by using DEFERRABLE clause.
- Once you've added a constraint, you cannot change it to DEFERRABLE. You must drop and recreate the constraint.
- When you add a DEFERRABLE constraint, you can mark it as INITIALLY IMMEDIATE or INITIALLY DEFERRED.
- INITIALLY IMMEDIATE means that the constraint is checked whenever you add, update, or delete rows from a table.
- INITIALLY DEFERRED means that the constraint is only checked when a transaction is committed.

Let's Drop and Create again :

```
alter table tab2 drop constraint tab2_fk;
alter table tab1 drop constraint tab1_fk;
alter table tab1 add constraint tab1_fk foreign key (c) references tab2(c) deferrable initially deferred;
alter table tab2 add constraint tab2_fk foreign key (c) references tab1(c) deferrable initially deferred;
```

```
insert into tab1 values ('A');
insert into tab1 values ('B');
insert into tab1 values ('C');
insert into tab2 values ('A');
```

insert into tab2 values ('B');
insert into tab2 values ('C');

select * from tab1

C
A
B
C

3 rows selected.

select * from tab2

✓ DONE
inserted

C
A
B
C

3 rows selected.